Internship Weekly Progress Report

Week 2

Intern Name	Ansh Rohilla	**Enrollment No.**	ADT24SOCB0169
Programme	B.Tech CSE (AI & Edge Computing)	**Year / Sem**	2nd Year, Sem 4
University	MIT-ADT University, Pune	**CGPA**	9.87
Organization	Vyana Innovations Pvt. Ltd.	**Department**	Strategy & Technology
Designation	AI & Business Intelligence Intern	**Mode**	Remote
Mentor	Anshika Rohilla, HR	**Report Period**	8 June – 14 June 2026
Project Title	AI-Powered Channel Partner & Lead Intelligence Dashboard		

This report documents the activities, deliverables, and learnings of Week 2 (8-14 June 2026) of the internship at Vyana Innovations Private Limited. The week was dedicated entirely to building the Flask REST API backend. This included setting up the SQLAlchemy database models, implementing the CSV/Excel data ingestion pipeline, building all partner and lead management endpoints, creating the analytics aggregation APIs, writing Pytest unit tests, and producing Postman/Swagger API documentation. All seven days were completed as planned with no blockers or delays.

Week 2 — At a Glance

Day	Date	Focus Area	Key Deliverable
Day 8	8 Jun	**Flask Architecture & DB Setup**	Flask blueprints skeleton + SQLAlchemy models
Day 9	9 Jun	**Ingestion Pipeline & Validation**	CSV/Excel upload API + Pandas validation
Day 10	10 Jun	**Partner Management APIs**	CRUD endpoints + tier classification logic
Day 11	11 Jun	**Lead Pipeline tracking APIs**	Kanban state transitions + lead score updates
Day 12	12 Jun	**Analytics Aggregation APIs**	Metrics summary, trends, and regional APIs
Day 13	13 Jun	**Testing & Documentation**	10 Pytest unit tests + Postman API collection
Day 14	14 Jun	**Mentor Review & Week 2 Close**	Sign-off received + Week 3 plan finalized

Day 8 | **Flask Architecture & DB Setup** | Monday, 8 June 2026

TASKS COMPLETED

- Attended Sprint 2 kickoff meeting with the mentor to align on the 17 REST API endpoint specifications
- Initialized the Flask application factory pattern (create_app) and structured app configurations in app/config.py
- Defined SQLAlchemy database schemas and relationships for the Partner and Lead models
- Set up blueprint routing, registering routes for /partners, /leads, /analytics, and /upload modules
- Configured database tables auto-creation logic (db.create_all()) within the application context

DELIVERABLES

- ✓ Flask application factory structure and database initialization setup
- ✓ SQLAlchemy schemas for Partner and Lead models with column definitions and constraints
- ✓ Blueprint routing skeleton with all base route handlers mapped

REFLECTION & KEY LEARNINGS

"Setting up the database models and application factory was an important architectural step. Designing relationships in SQLAlchemy is always satisfying, especially defining how leads cascade or reference their partners. Registering blueprints right away keeps the code modular from day one and prepares us for team growth."

Day 9 | **Ingestion Pipeline & Validation** | Tuesday, 9 June 2026

TASKS COMPLETED

- Built file upload endpoints (/api/upload/partners and /api/upload/leads) to process multipart CSV/Excel files
- Integrated Pandas for raw data parsing, null value treatment, and date formatting
- Coded data validation schemas using Marshmallow to enforce field types, lengths, and email formats
- Implemented database transaction controls to roll back commits if CSV validation failures occur
- Handled custom error responses for invalid files, empty uploads, and duplicate headers

DELIVERABLES

- ✓ CSV/Excel upload REST endpoints for partners and leads data
- ✓ Pandas parsing and data cleaning pipeline functions
- ✓ Marshmallow request schemas for validation

REFLECTION & KEY LEARNINGS

"Data ingestion is often where real-world applications fail, so handling edge cases was my main focus. I learned that sanitizing user uploads—especially dates and emails—requires strict validation to avoid database corruption down the line. Seeing the Pandas-to-SQLAlchemy transition work cleanly was a great milestone."

Day 10 | **Partner Management APIs** | Wednesday, 10 June 2026

TASKS COMPLETED

- Implemented GET /api/partners route to retrieve channel partners with dynamic query parameters
- Coded search indexing (by company name) and filter variables (by tier, region, and status)
- Built database sorting (by revenue, deal count, name) and server-side pagination wrapper
- Implemented GET /api/partners/ route returning details and chronological lead activity logs
- Created POST, PUT, and DELETE partner management endpoints

DELIVERABLES

✓ Complete Partner CRUD endpoints in backend/app/routes/partners.py

✓ Partner query filtration, sorting, and pagination logic

✓ Active partner database seeder script

REFLECTION & KEY LEARNINGS

"Developing the partner endpoints taught me how critical structured API design is. Fetching a partner should not just yield their name; attaching their complete lead timeline makes the UI much more interactive and useful. Optimizing query joins in SQLAlchemy keeps database latencies minimal."

Day 11 | **Lead Pipeline tracking APIs** | Thursday, 11 June 2026

TASKS COMPLETED

- Implemented leads retrieval endpoints (GET /api/leads and GET /api/leads/)
- Coded pipeline status transition handler via PUT /api/leads/ to support Kanban board movements
- Configured automatic ML score recalculation triggers on lead detail modifications
- Implemented partner lead assignments and reassignment matches
- Built lead creation (POST) and deletion (DELETE) endpoints

DELIVERABLES

✓ Complete Lead CRUD endpoints in backend/app/routes/leads.py

✓ Lead status transition handler (Kanban stage update route)

✓ Automatic lead scoring trigger integration

REFLECTION & KEY LEARNINGS

"Managing lead state transitions represents the core operations of the dashboard. Building the update endpoint to automatically recalculate the lead's conversion probability before writing to SQLite felt like bringing the dashboard to life. Emphasizing clean status transitions prevents invalid state changes."

Day 12 | **Analytics Aggregation APIs** | Friday, 12 June 2026

TASKS COMPLETED

- Programmed aggregate endpoints: /api/analytics/summary (KPI counts: active partners, leads, conversion rates, pipeline value)
- Coded /api/analytics/regional returning lead/partner aggregates grouped by region
- Developed /api/analytics/trends to compute weekly lead velocity trends
- Integrated Flask-Caching (using SimpleCache) to store analytical metrics for 5 minutes, boosting speed
- Implemented caching invalidation triggers on lead create/update/delete requests

DELIVERABLES

- ✓ Summary, regional, and weekly trend endpoints
- ✓ Optimized database aggregation queries
- ✓ Cache management and cache-invalidation functions

REFLECTION & KEY LEARNINGS

"Writing efficient aggregation queries is essential for fast dashboards. Performing heavy calculations (like total pipeline value) directly in SQL is much faster than processing rows in Python. Adding Flask-Caching ensures subsequent dashboard reloads load in milliseconds rather than seconds."

Day 13 **Testing & Documentation** Saturday, 13 June 2026

TASKS COMPLETED

- Authored an automated unit test suite inside backend/tests/test_endpoints.py containing 10 Pytest cases
- Tested payload boundaries, validation errors, and transactional rollbacks
- Compiled all REST API configurations into a Postman Collection JSON schema with mock requests
- Documented API setup, startup commands, and request formats in the project README

DELIVERABLES

- ✓ Pytest suite with 10 automated backend test cases
- ✓ Postman API Collection JSON configuration schema
- ✓ Completed REST API documentation in README.md

REFLECTION & KEY LEARNINGS

"Writing tests before the final review is a great safety net. I caught two minor validation bugs in my lead creation endpoint before they could cause issues. Having a fully documented Postman collection also makes testing endpoints manually much easier and acts as a clear reference for the frontend."

Day 14	Mentor Review & Sprint Close	Sunday, 14 June 2026

TASKS COMPLETED

- Conduct the Week 2 milestone progress check-in meeting with the mentor
- Demonstrate all 17 REST API endpoints using Postman (uploads, CRUD queries, cached analytics, error handling)
- Receive sign-off on API design, request-response schemas, and error shapes
- Structure the Week 2 progress report and planned the Week 3 sprint

DELIVERABLES

- ✓ Mentor sign-off on Flask REST API architecture
- ✓ Git commits pushed to the remote repository
- ✓ Weekly progress report compiled as PDF

REFLECTION & KEY LEARNINGS

"Demonstrating a functional API backend to the mentor was very rewarding. Seeing the upload endpoints process CSV files and populate the database instantly showed that the core engine of the project is solid. The mentor's feedback to return human-friendly error messages helped polish our error responses."

Week 2 — Summary Report

1. Overview

Week 2 of the internship at Vyana Innovations Private Limited was focused entirely on backend engineering and REST API development. Following our architectural blueprints, we built a fully operational Flask API service, established SQLAlchemy database schemas with auto-seeding, created robust data validation pipelines, and integrated caching for heavy analytics. All activities were completed on schedule, providing a solid, tested backend foundation ready to support our machine learning and frontend dashboard sprints.

2. Key Accomplishments

Accomplishment Area	Summary of Achievements
Development Environment	Refactored virtual env dependencies, added Gunicorn support for Unix deployments, and structured environment variable configurations.
Database Setup	Initialized SQLite schemas, created SQLAlchemy ORM models (Partner, Lead), and set up auto-seeding logic triggered on application startup.
API Development	Coded and integrated 17 endpoints covering partner management, lead directories, regional calculations, and data syncs.
Ingestion & Validation	Designed file ingestion routes using Pandas for data cleansing, date formatting, and Marshmallow for strict field schema validation.
Testing & Documentation	Authored a backend unit testing suite containing 10 Pytest test cases, and compiled a Postman API collection for developer reference.

3. Key Learnings & Professional Development

- **Flask Factory Pattern**: Structuring applications via factory methods simplifies extension management and enables isolated route registry through blueprints.
- **Data Sanitation at the Edge**: Standardizing and validating incoming user uploads (CSV/Excel) using Marshmallow prevents database anomalies and application exceptions.
- **ORM Relationship Mappings**: Utilizing SQLAlchemy cascades and relational joins enables fast traversal of partner-lead activity timelines with optimal query performance.
- **Automated Test Automation**: Writing Pytest scripts for endpoints ensures that subsequent refactoring or deployment tasks do not introduce regressions.

• **API Response Consistency**: Returning uniform JSON response envelopes (success, data, pagination, message) simplifies client-side Axios consumption.

• **Caching Invalidation**: Implementing cache clearance triggers on lead/partner write requests is essential to maintain data consistency on overview statistics.

4. Deliverables Status

#	Deliverable	Status
1	Database factory setup & SQLAlchemy models	**Complete**
2	Marshmallow schemas & request validation	**Complete**
3	CSV/Excel file upload ingestion routes	**Complete**
4	Partner directory CRUD endpoints	**Complete**
5	Lead pipeline management endpoints (Kanban update)	**Complete**
6	Analytics summary, regional, and trends endpoints	**Complete**
7	Flask-Caching query optimization integration	**Complete**
8	Pytest suite with 10 automated unit tests	**Complete**
9	Postman Collection REST API documentation	**Complete**
10	Week 2 mentor review sign-off and git push	**Complete**

5. Week 3 Plan

Week 3 (15-21 June 2026) will focus on **Phase 3: Machine Learning Model Development**. This includes performing Exploratory Data Analysis (EDA) on channel partner historical leads, designing feature engineering transformers, comparing multiple classification models (Logistic Regression, Random Forest, XGBoost), performing GridSearchCV hyperparameter tuning, exposing API inference endpoints, and drafting model performance diagnostics.

6. Overall Reflection

Week 2 shifted my focus from project scoping and planning to core software engineering. Building database models, writing clean query joins, and sanitizing user inputs required meticulous attention to detail. This week reinforced that backend development is not just about writing code that works, but about designing scalable, secure, and performant APIs. I feel highly confident in our backend foundation as we pivot to the machine learning sprint in Week 3.